Movie Watchlist — REST API Mini Project

Full-stack web application using Node.js, Express.js, HTML, CSS, and JavaScript

1. Project Objective

This project demonstrates a small full-stack web application with a REST API. Users can create, read, update, delete, and filter movies through a browser interface without page reloads.

2. Technologies

• Node.js
• Express.js
• HTML5 / CSS3 / JavaScript
• Fetch API
• In-memory data storage

3. Project Structure

server/server.js — Express REST API
public/index.html — frontend page
public/style.css — UI styling
public/script.js — frontend Fetch/CRUD logic
README.md — setup and API documentation
REPORT.pdf — project report and evidence

4. REST API Design

Method	Endpoint	Purpose	Status
GET	/api/movies	Get all movies; supports genre/status query filters	200
GET	/api/movies/:id	Get one movie by ID	200 / 404
POST	/api/movies	Create a movie with validation	201 / 400
PATCH	/api/movies/:id	Update a movie	200 / 400 / 404
DELETE	/api/movies/:id	Delete a movie	204 / 404

5. Query-string Filtering

The GET collection endpoint supports filters such as `/api/movies?genre=Sport` and `/api/movies?status=Watched`. The frontend sends these filters using fetch() and updates the movie list without a page reload.

6. Validation

Required fields are checked on POST/PATCH. Year is restricted to a reasonable range, rating must be between 0 and 10, and status must be either Watched or Want to Watch. Invalid input returns HTTP 400; a missing movie returns HTTP 404.

7. Initial Movie Data

Movie	Year	Genre	Status	Rating
Venom	2018	Action / Sci-Fi	Watched	10/10

Movie	Year	Genre	Status	Rating
Venom: Let There Be Carnage	2021	Action / Sci-Fi	Watched	10/10
Venom: The Last Dance	2024	Action / Sci-Fi	Watched	8/10
Fast & Furious 7	2015	Action	Watched	8/10
Gran Turismo	2023	Action / Sport	Watched	9/10

8. Frontend CRUD and Debugging Evidence

The following screenshots were taken from the running application and show the interface plus successful update/filter behavior.

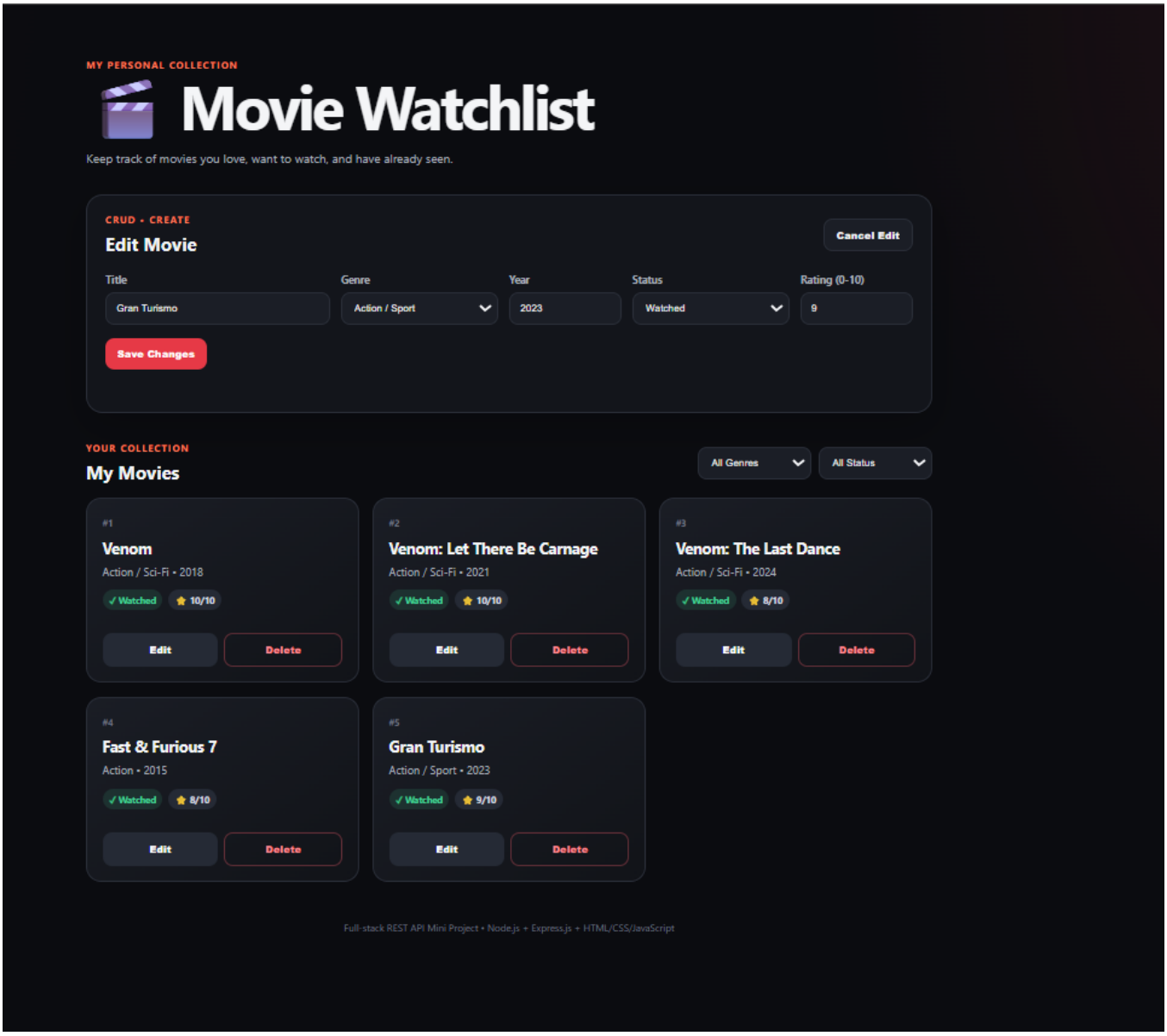


Figure 1 — Main Movie Watchlist with movie cards and CRUD controls

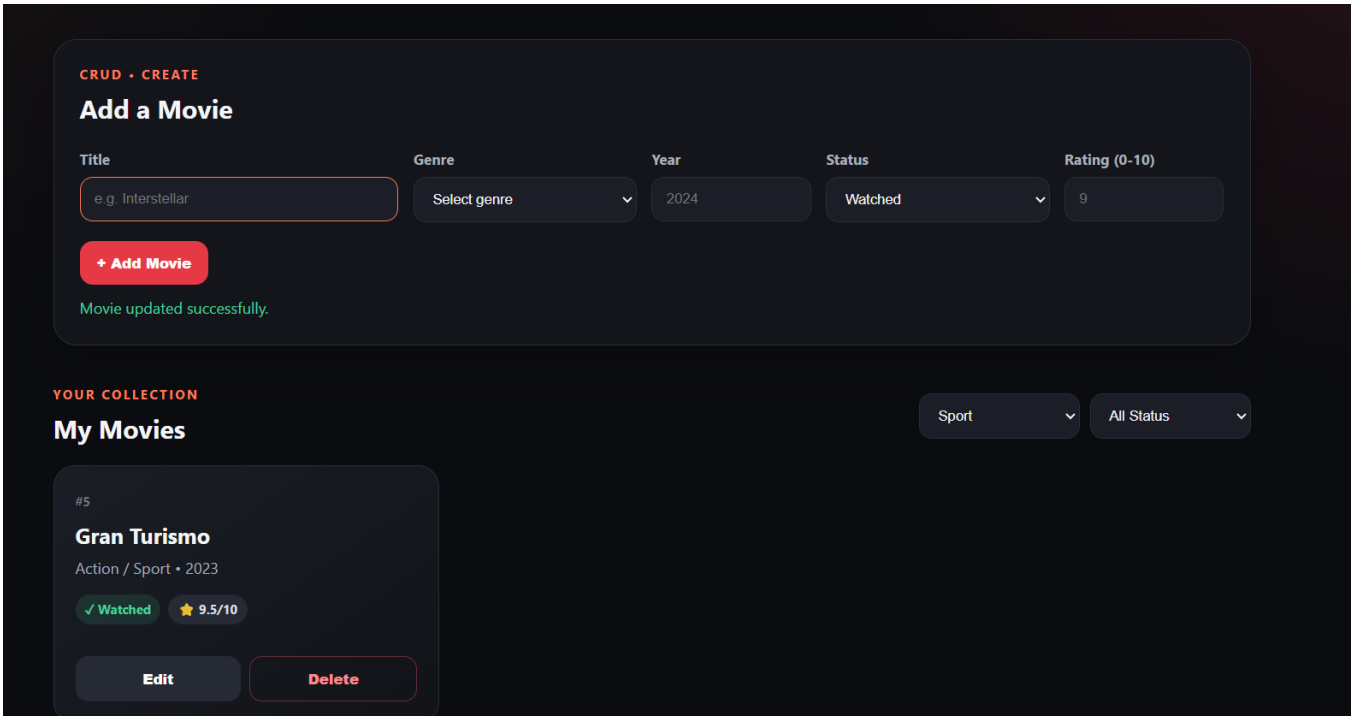


Figure 2 — Successful update message and Sport genre query filter showing Gran Turismo

9. How to Run

1) Open the project folder in VS Code.
2) Run `npm install`.
3) Run `npm run dev`.
4) Open `http://localhost:3000` in a browser.

10. Conclusion

The Movie Watchlist satisfies the mini-project requirements for an Express.js REST API, full CRUD operations, query-string filtering, input validation/status codes, and a frontend that communicates with the student's own API using JavaScript fetch().